Search

Gradle User Manual
Getting Started

Gradle Tutorials
▸ Beginner Tutorial
▸ Intermediate Tutorial
▸ Advanced Tutorial

Gradle Releases
All Releases
Release Notes
Installing Gradle
▾ Upgrading Gradle
Within versions 9.x.y
To version 9.0.0
Within versions 8.x
From version 7.x to 8.0
From version 6.x to 7.0
From version 5.x to 6.0
From version 4.x to 5.0
▸ Migrating to Gradle
Compatibility Notes
Gradle's Feature Lifecycle

Gradle Fundamentals
▸ Learning Gradle Basics
▸ Writing Build Scripts
▸ Creating Plugins

Gradle Reference
▸ Runtime and Configuration
▸ DSLs and APIs
▸ Gradle Managed Types
▸ Tasks
▸ Plugins
▸ Dependencies
▸ Platforms
▸ Best Practices
▸ Other Topics

Structuring Gradle Builds
Organizing Projects
Multi-Project Builds
Sharing Build Logic
Composite Builds
Configuration on Demand
Isolated Projects

Optimizing Gradle Builds
Improving Performance
▸ Build Cache
▸ Configuration Cache

Securing Gradle Builds
Supply Chain Security
Verifying Dependencies

IDE & Tool Integration
Third-party Tools
▸ APIs

How-To-Guides
▸ Structuring Builds
▸ Dependency Management

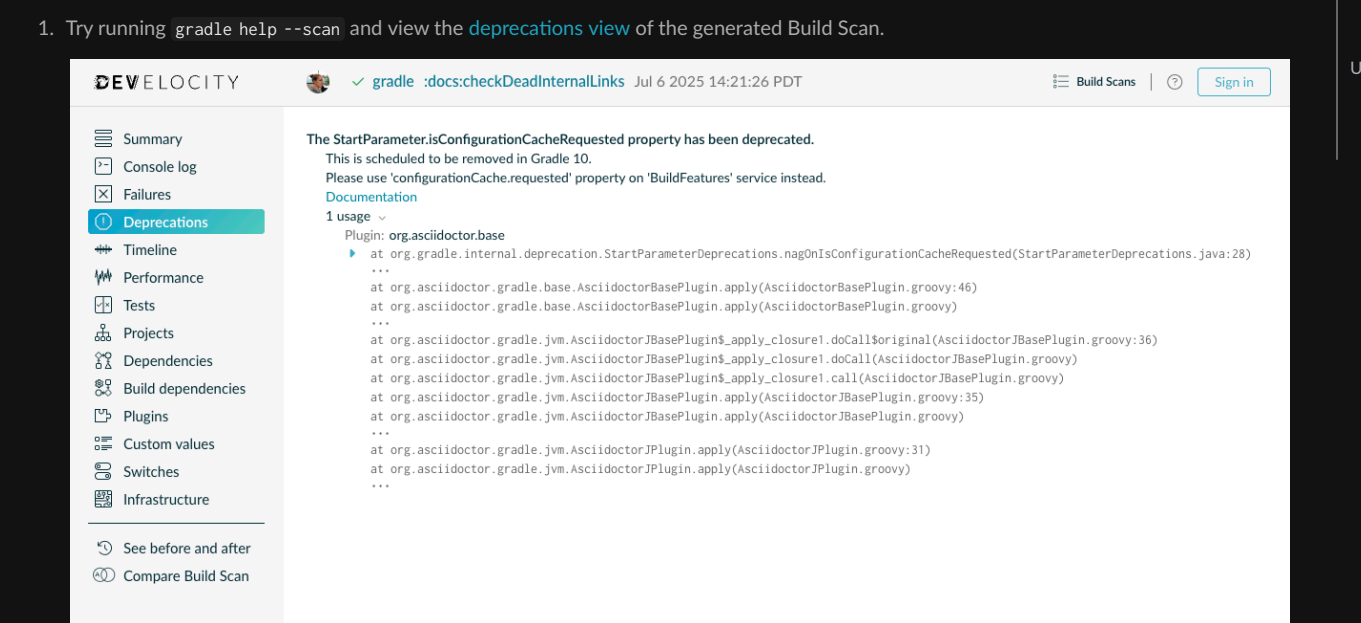
More Resources
Samples
Glossary
Single Page Version

Upgrading within Gradle 9.x.y

This chapter provides the information you need to migrate your Gradle 9.x.y builds to the latest. For migrating to Gradle 9.0.0, see the older migration guide first.

We recommend the following steps for all users:

1. Try running `gradle help --scan` and view the deprecations view of the generated Build Scan.

   The StartParameter.isConfigurationCacheRequested property has been deprecated.
   This is scheduled to be removed in Gradle 10.
   Please use 'configurationCache.requested' property on 'BuildFeatures' service instead.
   Documentation
   1 usage
   Plugin: org.asciidoctor.base

   ```
   at org.gradle.internal.deprecation.StartParameterDeprecations.nagOnIsConfigurationCacheRequested(StartParameterDeprecations.java:28)
   ...
   at org.asciidoctor.gradle.base.AsciidoctorBasePlugin.apply(AsciidoctorBasePlugin.groovy:46)
   at org.asciidoctor.gradle.base.AsciidoctorBasePlugin.apply(AsciidoctorBasePlugin.groovy)
   ...
   at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin$_apply_closure1.doCall$original(AsciidoctorJBasePlugin.groovy:36)
   at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin$_apply_closure1.doCall(AsciidoctorJBasePlugin.groovy)
   at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin$_apply_closure1.call(AsciidoctorJBasePlugin.groovy)
   at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin.apply(AsciidoctorJBasePlugin.groovy:35)
   at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin.apply(AsciidoctorJBasePlugin.groovy)
   ...
   at org.asciidoctor.gradle.jvm.AsciidoctorJPlugin.apply(AsciidoctorJPlugin.groovy:31)
   at org.asciidoctor.gradle.jvm.AsciidoctorJPlugin.apply(AsciidoctorJPlugin.groovy)
   ...
   ```

   This lets you see any deprecation warnings that apply to your build.

   Alternatively, you can run `gradle help --warning-mode=all` to see the deprecations in the console, though it may not report as much detailed information.

2. Update your plugins.

   Some plugins will break with this new version of Gradle because they use internal APIs that have been removed or changed. The previous step will help you identify potential problems by issuing deprecation warnings when a plugin tries to use a deprecated part of the API.

3. Run `gradle wrapper --gradle-version 9.4.0` to update the project to 9.4.0.
4. Try to run the project and debug any errors using the Troubleshooting Guide.

On this Page
Upgrading from 9.3.0 and earlier
Potential breaking changes
Deprecations
Upgrading from 9.2.0 and earlier
Potential breaking changes
Deprecations
Upgrading from 9.1.0 and earlier
Potential breaking changes
Deprecations
Upgrading from 9.0.0 and earlier
Potential breaking changes
Deprecations

Upgrading from 9.3.0 and earlier

Potential breaking changes

`ProjectBuilder` now enforces consistent build-scoped locations

`ProjectBuilder` allows you to configure a specific project directory for tests. While `project.projectDir` and `project.rootDir` have always respected this setting, `project.layout.settingsDirectory` previously did not. This discrepancy could cause file resolution to inadvertently escape the project directory.

Gradle now anchors the settings search directly to the configured project directory. This ensures that `layout.settingsDirectory`, `projectDir`, and `rootDir` all point to the same consistent location. Projects created via `ProjectBuilder` are now better isolated from the host build environment.

If your tests specifically relied on `layout.settingsDirectory` pointing to an external location, they will need to be adjusted. Even if you do not use `settingsDirectory` directly, you may still observe changes in file resolution. Previously, dependency management files could be "leaked" into the test from the host build environment; this is no longer the case.

For more details see the related issue.

The `java-gradle-plugin` plugin now adds the `gradleApi()` dependency to the `compileOnlyApi` scope

The `gradleApi()` dependency is now added to the `compileOnlyApi` scope instead of the `api` scope. This prevents the `gradleApi()` from leaking into the runtime classpath of other dependents of the plugin project.

This might break projects that were implicitly relying on the `gradleApi()` being on the compilation or runtime classpath. If this is the case, add a `gradleApi()` dependency to the appropriate scope to restore the previous behavior.

For most plugin projects, this change should be transparent as:

- at compilation time, the plugin project will get the `gradleApi()`
- the default test source set will also automatically get the `gradleApi()` dependency on the compilation and runtime classpaths

If any additional test source set is used (e.g., integration tests), the `plugin` extension offers the `plugins.testSourceSet` method to register the source set for automatic management. If this cannot be done, a regular declaration of the `gradleApi()` dependency on the test source can be used as well.

Stricter validation for published plugins

For plugin builds that apply any of the `com.gradle.plugin-publish`, `ivy-publish`, or `maven-publish` plugins, Gradle now automatically enables stricter validation of plugin code.

In order not to break your builds, this does not apply to local plugins (in `buildSrc` or included builds containing build logic). However, we encourage you to always enable stricter validation:

build.gradle.kts

```kotlin
tasks.validatePlugins {
    enableStricterValidation = true
}
```

CodeNarc compilation classpath is set by default

The CodeNarc plugin now automatically populates the `compilationClasspath` of a `CodeNarc` task with the compile classpath of its associated source set.

CodeNarc offers Enhanced Classpath Rules (like `UnusedImport` or `DuplicateImport`) that require the project's compiled classes and dependencies to be analyzed for full accuracy. Previously, you had to wire this up manually. Now, it works out of the box.

This change introduces a task dependency. The `CodeNarc` task must now wait for the `compile` task to finish so it can access the compiled classes.

If you do not use enhanced rules and want to restore parallel execution, you can manually empty the `compilationClasspath`:

build.gradle.kts

```kotlin
plugins {
    id("groovy")
    id("codenarc")
}

tasks.withType<CodeNarc>().configureEach {
    // Override the default compilation classpath
    compilationClasspath = files()
}
```

If your build relies on a custom configuration for the `compilationClasspath` of a `CodeNarc` task, you will need to continue explicitly setting it to override the new default behavior.

System property priority for Wrapper execution has changed

The priority of system properties passed to the Gradle Wrapper now correctly follows the documented order of precedence. In previous versions, properties defined in `gradle.properties` files could unexpectedly override those passed via the command line.

If a property is defined in multiple locations, Gradle now strictly honors the following hierarchy (from highest to lowest):

Priority	Source	Example
1 (Highest)	Command Line Option	`./gradlew build -Dproperty.name=value`
2	Gradle User Home	`~/.gradle/gradle.properties`
3 (Lowest)	Project Directory	`[project-root]/gradle.properties`

For more details see the related issue.

Upgrade to Kotlin 2.3.0

The embedded Kotlin has been upgraded from 2.2.21 to Kotlin 2.3.0.

Upgrade to Zinc 1.12.0

Zinc has been updated to 1.12.0.

Deprecations

Deprecation of `DomainObjectCollection.findAll(Closure)`

The `findAll(Closure)` method on Gradle collections is now deprecated and scheduled for removal in Gradle 10.0.0.

This method relies specifically on Groovy types and eagerly evaluates the contents of the container.

To fix this, use the similar `DomainObjectCollection.matching(Spec)`. While not a direct replacement for `findAll`, `matching` is lazy, it returns a new collection that only filters elements as they are actually needed by the build:

```groovy
// Deprecated:
def checkTasks = tasks.findAll { it.name.startsWith("check") }

// Recommended:
def checkTasks = tasks.matching { it.name.startsWith("check") }
```

Deprecation of methods taking `Closure` on `Test` tasks

The following APIs are deprecated and will be removed in Gradle 10.0.0:

- `AbstractTestTask.onOutput(Closure)` can be replaced with `AbstractTestTask.addTestOutputListener(TestOutputListener)`
- `AbstractTestTask.beforeTest(Closure)` can be replaced with `AbstractTestTask.addTestListener(TestListener)`
- `AbstractTestTask.afterTest(Closure)` can be replaced with `AbstractTestTask.addTestListener(TestListener)`
- `AbstractTestTask.beforeSuite(Closure)` can be replaced with `AbstractTestTask.addTestListener(TestListener)`
- `AbstractTestTask.beforeSuite(Closure)` can be replaced with `AbstractTestTask.addTestListener(TestListener)`
- `Test.testFramework(Closure)` can be replaced with `Test.options(Action)`

Deprecation of `apply false` in precompiled script plugins

The use of `apply false` within precompiled script plugins is now deprecated and will result in an error in Gradle 10.0.0.

In a precompiled script, the `plugins {}` block behaves differently than in a standard build script. Currently, if you write `apply false`, Gradle applies the plugin anyway. This creates confusion because the syntax suggests you are merely adding a plugin to the classpath without activating it, which is not what actually happens.

The fix depends on whether you actually want the plugin to be active in your precompiled script:

- If you want to use the plugin: Remove the `apply false` statement.
- If you do NOT want to use the plugin: Delete the line entirely.

my-plugin.gradle.kts

```kotlin
plugins {
    // Deprecated (and misleading, as it is still applied)
    id("org.gradle.test-retry") apply false
    // The plugin will still be on the classpath,
    // but it will not be applied as part of this precompiled script plugin.

    // Recommended: Either remove 'apply false' to keep using it,
    // or delete the line to stop using it.
    id("org.gradle.test-retry")
}
```

Deprecation of `version` in precompiled Settings script plugins

The use of `version` within precompiled Settings script plugins is now deprecated and will become an error in Gradle 10.0.0.

In precompiled scripts, `version` has no effect. The plugin version is already fixed by the script's own build file; declaring it again inside the script is ignored and causes confusion about which version is actually in use.

To fix this, remove the `.version()` or `version "…"` call from the `plugins {}` block:

my-plugin.settings.gradle.kts

```kotlin
plugins {
    // Deprecated:
    id("org.gradle.test-retry") version "x.y.z"
    // Recommended:
    id("org.gradle.test-retry")
}
```

Upgrading from 9.2.0 and earlier

Potential breaking changes

Referential equality is not guaranteed for `Project` instances

Instances of `Project` type representing the same logical Gradle project do not provide any guarantee of being equal by reference (`==` in Java, `===` in Kotlin). However, historically it was possible to observe that such `Project` instances were exactly the same.

In Gradle 9.3.0, `Project` instances (for the same logical project) **can be different** with respect to the referential equality, even if obtained within the same context, e.g., in the same build script. This change is necessary to facilitate future performance improvements of Gradle. The `Project.equals()` equality behavior remains unchanged.

Avoid referential equality in Kotlin:

build.gradle.kts

```kotlin
// DON'T do this
project.rootProject === project.parent
```

Avoid referential equality in Groovy:

build.gradle

```groovy
// DON'T do this
project.rootProject.is(project.parent)
```

Avoid referential equality in Java:

MyPlugin.java

```java
// DON'T do this
project.getRootProject() == project.getParent();
```

In general, it is better to check project equality via `Project.getPath()` or `Project.getBuildTreePath()` for composite-build support. The paths are also better suited to be keys in data structures, like maps.

TestNG output may change when using versions before 6.9.13.3

As part of the `AbstractTestTask` refactoring, Gradle's integration with TestNG has been updated. Gradle now relies on a correctly functioning `IClassListener` to report the hierarchy of test classes and methods.

When using TestNG versions **earlier than 6.9.13.3**, this can lead to different or degraded output:

- With older versions, class information may be lost. For example, output that previously looked like: `org.gradle > TestClass > ok` may now be reported simply as: `ok`.
- For TestNG versions **from 6.9.10 up to (but not including) 6.9.13.3**, the `IClassListener` API exists but is broken. This can result in even worse output, such as empty or missing names.

To get correct and stable output, we recommend upgrading to **TestNG 6.9.13.3 or newer**.

Upgrade to Kotlin 2.2.21

The embedded Kotlin has been upgraded from 2.2.20 to 2.2.21.

Upgrade to Jansi 2.4.2

Jansi was upgraded from 1.18 to 2.4.2 to pick up support for Windows ARM64.

Upgrade to ASM 9.9

ASM was upgraded from 9.8 to 9.9 to ensure earlier compatibility for Java 26.

Upgrade to Groovy 4.0.29

Groovy has been updated to 4.0.29.

Upgrade to JaCoCo 0.8.14

JaCoCo has been updated to 0.8.14.

Deprecations

Deprecation of the `Wrapper.getAvailableDistributionTypes()` method

The method on the `Wrapper` task has been deprecated and will be removed in Gradle 10.

Use `Wrapper.DistributionType.values()` to obtain the available distribution types instead.

Deprecation of publishing dependencies on unpublished projects

When publishing a project, Gradle resolves project dependencies to the coordinates of the target project's publication. If the target project has no publication, Gradle currently resolves the dependency silently using that project's `group`, `name`, and `version`.

Starting with Gradle 10, this behavior is deprecated. Gradle will no longer silently ignore the absence of a publication. Publishing a project that depends on another project without a publication will be forbidden and will cause the build to fail. This change prevents publishing broken metadata with dependency coordinates that cannot be resolved.

The example below demonstrates a build that triggers the deprecated behavior:

build.gradle.kts

```kotlin
plugins {
    id("java-library")
    id("maven-publish")
}

group = "com.example"
version = "1.0.0"

dependencies {
    api(project(":other"))
}

publishing {
    publications {
        create<MavenPublication>("maven") {
            from(components["java"])
        }
    }
}
```

other/build.gradle.kts

```kotlin
plugins {
    id("java-library")
}

group = "com.example"
version = "1.0.0"
```

To avoid this deprecation, ensure that all project dependencies of published projects are also published. In the example above, applying the `maven-publish` plugin and configuring a publication in the `:other` project resolves the issue:

other/build.gradle.kts

```kotlin
plugins {
    id("java-library")
    id("maven-publish")
}

group = "com.example"
version = "1.0.0"

publishing {
    publications {
        create<MavenPublication>("maven") {
            from(components["java"])
        }
    }
}
```

Deprecation of legacy `Usage` attribute values

Since Gradle 5.6, the `Usage` attribute has been split into an additional `LibraryElements` attribute. In the JVM ecosystem, `Usage` indicates whether a variant is intended for compilation or runtime, while `LibraryElements` specifies the format of the artifact (for example, a JAR file or a classes directory).

To ease migration, Gradle has automatically mapped legacy `Usage` values to their corresponding `Usage` and `LibraryElements` pairs:

Legacy `Usage`	Replaced `Usage`	Replaced `LibraryElements`
`java-api-jars`	`java-api`	`jar`
`java-api-classes`	`java-api`	`classes`
`java-runtime-jars`	`java-runtime`	`jar`
`java-runtime-classes`	`java-runtime`	`classes`
`java-runtime-resources`	`java-runtime`	`resources`

Starting with Gradle 10, this automatic mapping will no longer occur when legacy `Usage` values are added directly to an `AttributeContainer` in build logic. To maintain backward compatibility for already published modules, Gradle will continue translating legacy `Usage` values found in published Gradle Module Metadata.

Deprecation of using module coordinates to depend on the current project

Starting with Gradle 10, declaring a dependency on the current project using module coordinates (`group`, `name`, `version`) will no longer resolve to that project. Instead, Gradle will attempt to resolve that dependency from a repository.

The example below demonstrates the change in behavior:

my-project/build.gradle.kts

```kotlin
group = "com.example"
version = "1.0.0"

val deps = configurations.dependencyScope("deps")
val classpath = configurations.resolvable("classpath") {
    extendsFrom(deps.get())
    attributes.attribute(Category.CATEGORY_ATTRIBUTE, objects.named(Category.LIBRARY))
}

val elements = configurations.consumable("elements") {
    attributes.attribute(Category.CATEGORY_ATTRIBUTE, objects.named(Category.LIBRARY))
}

dependencies {
    // In Gradle 9.x, this dependency resolves to the `elements` configuration.
    // In Gradle 10, this dependency will attempt to resolve from a repository.
    deps("com.example:my-project:1.0.0")
}
```

To continue depending on the current project, use a `project` dependency:

my-project/build.gradle.kts

```kotlin
dependencies {
    // Declare a dependency on the current project to continue resolving
    // to the current project.
    deps(project)
}
```

Deprecation of `ModuleVersionSelector` to `ModuleComponentSelector` conversion

The conversion of `ModuleVersionSelector` to `ModuleComponentSelector` (used in `ResolutionStrategy.force(…)`, `DependencyResolveDetails.useTarget(Object)`, and `PluginResolveDetails.useModule(Object)`) has been deprecated and will be removed in Gradle 10.0.0.

Typically, `ModuleVersionSelector` instances are `DependencyContstraint` objects.

Note that this deprecation does not apply to `ExternalDependency` objects, despite them implementing `ModuleVersionSelector`.

To fix this deprecation, pass one of the supported notations (for example, a String in the `group:name:version` format).

Upgrading from 9.1.0 and earlier

Potential breaking changes

Upgrade to Kotlin 2.2.20

The embedded Kotlin has been upgraded from 2.2.0 to Kotlin 2.2.20.

Removed incubating `ObjectFactory#dependencyCollector()` method

The incubating `ObjectFactory#dependencyCollector()` method has been removed. You can still create `DependencyCollectors` within Gradle managed types.

Consumable configurations in bundled plugins are now initialized lazily

Consumable configurations created by bundled Gradle plugins are now initialized only when needed. `Configure` actions on these configurations no longer run at configuration time by default. They only execute if the configuration is published, consumed as a variant, or otherwise realized by build logic.

For example:

build.gradle.kts

```kotlin
plugins {
    id("java-library")
}

configurations.named("apiElements").configure {
    println("Configuring apiElements")
}
```

With this change, the `Configuring apiElements` line is no longer printed during configuration time unless `apiElements` is actually realized.

See Declaring Configurations for more guidance.

`ValidatePlugins` now has stricter Java version requirements

The `ValidatePlugins` task must now run on a Java version that is supported by the Gradle daemon. This change was made because the task depends on several core Gradle services, which may now be compiled to the same bytecode version supported by the daemon.

By default, the task's convention has been updated:

- If your project's toolchain is compatible, `ValidatePlugins` will use it.
- Otherwise, it will fall back to the Java version used to run Gradle.

If you explicitly set a toolchain like this:

Kotlin Groovy

`build.gradle`

```groovy
tasks.withType(ValidatePlugins).configureEach {
    javaLauncher.set(
        project.javaToolchains.launcherFor {
            languageVersion.set(JavaLanguageVersion.of(17))
        }
    )
}
```

If the specified Java version is **not** compatible with the Gradle daemon, you must update it to a compatible version.

Deprecations

Deprecation of `Project.container(…)` methods

The `Project.container(…)` methods are deprecated and will be removed in Gradle 10. These methods manually create named domain object containers.

Use a managed property to let Gradle instantiate containers automatically. If a managed property isn't possible, use `ObjectFactory.domainObjectContainer(…)` (available since Gradle 5.5). Unlike `Project.container(Class)`, the `ObjectFactory` version decorates container elements and makes them extension aware.

Deprecation of ruleSource-based dependency management APIs

The `RuleSource`-based dependency management APIs have been deprecated and will be removed in Gradle 10.0.0.

Deprecated APIs include:

- `ComponentMetadataHandler.all(Object)`
- `ComponentMetadataHandler.withModule(Object,Object)`
- `ComponentSelectionRules.all(Object)`
- `ComponentSelectionRules.withModule(Object,Object)`

Use the alternative methods that accept a `ComponentMetadataRule` class or an `Action`.

Deprecation of calling `registerFeature` without applying the Java plugin

Creating a JVM feature with `JavaPluginExtension#registerFeature` before applying the Java plugin has been deprecated and will become an error in Gradle 10.0.0.

Ensure the Java plugin is applied before invoking `registerFeature`. The following bundled plugins apply the Java plugin automatically:

- `java-library`
- `application`
- `groovy`
- `scala`
- `war`

Upgrading from 9.0.0 and earlier

Potential breaking changes

Upgrade to ASM 9.8

ASM was upgraded from 9.7.1 to 9.8 to ensure earlier compatibility for Java 25.

Upgrade to Groovy 4.0.28

Groovy has been updated to Groovy 4.0.28.

Deprecations

Deprecation of multi-string dependency notation

In an effort to simplify and standardize the Gradle API, the multi-string dependency notation used in dependency management has been deprecated and will no longer be permitted in Gradle 10. Gradle will primarily accept dependency declarations in the form of a single string, with each dependency coordinate separated by a colon.

Below are examples of the deprecated multi-string notation:

Kotlin Groovy

`build.gradle`

```groovy
dependencies {
    implementation(group: 'org', name: 'foo', version: '1.0')
    implementation(group: 'org', name: 'foo', version: '1.0', configuration: 'conf')
    implementation(group: 'org', name: 'foo', version: '1.0', classifier: 'classifier')
    implementation(group: 'org', name: 'foo', version: '1.0', ext: 'ext')
}

testing.suites.test {
    dependencies {
        implementation(module(group: 'org', name: 'foo', version: '1.0'))
    }
}
```

These declarations should be replaced with the single-string notation:

Kotlin Groovy

`build.gradle`

```groovy
dependencies {
    implementation("org:foo:1.0")
    implementation("org:foo:1.0") {
        targetConfiguration = "conf"
    }
```

```
    implementation("org:foo:1.0:classifier")
    implementation("org:foo:1.0@ext")
}

testing.suites.test {
    dependencies {
        implementation("org:foo:1.0")
    }
}
```

In some cases, a complete single-string notation may not be known up front. Instead of concatenating the coordinates into a new string, it is possible to use a `DependencyFactory` to create `Dependency` instances directly from the individual components:

Kotlin Groovy

build.gradle

```groovy
def group = "org"
def artifactId = "foo"
def version = "1.0"

configurations.dependencyScope("implementation") {
    dependencies.add(project.dependencyFactory.create(group, artifactId, version))
}
```

Deprecation of `ReportingExtension.file(String)`

The `file()` method on `ReportingExtension` has been deprecated and will be removed in Gradle 10.0.0.

Instead, use `ReportingExtension.getBaseDirectory()` with `file(String)` or `dir(String)`.

Deprecation of `ReportingExtension.getApiDocTitle()`

The `getApiDocTitle()` method on `ReportingExtension` has been deprecated and will be removed in Gradle 10.0.0.

There is no direct replacement for this method.

Deprecation of `JavaForkOptions.setAllJvmArgs()`

The `setAllJvmArgs()` method on `JavaForkOptions` and, by inheritance, on `JavaExecSpec` has been deprecated and will be removed in Gradle 10.0.0.

Instead, to overwrite existing JVM arguments, use:

- `JavaForkOptions.jvmArgs()`
- `JavaForkOptions.setJvmArgs()`
- Provide a `CommandLineArgumentProvider` to add arguments via `JavaForkOptions.getJvmArgumentProviders()`

Note that `setAllJvmArgs()` method on `JavaForkOptions` cleared all fork options before setting `jvmArgs`. The properties cleared included:

- System properties configured via `JavaForkOptions.systemProperties`
- JVM argument providers configured via `JavaForkOptions.jvmArgumentProviders`
- Argument providers configured via `JavaExecSpec.argumentProviders`
- Memory settings configured via `JavaForkOptions.minHeapSize` and `JavaForkOptions.maxHeapSize`
- All other JVM arguments configured via `JavaForkOptions.jvmArgs`
- The assertion and debug flags configured via `JavaForkOptions.enableAssertions` and `JavaForkOptions.debug`

If the arguments you provide to `setJvmArgs()` or `jvmArgs()` depend on any of the above properties being cleared, you will need to manually clear them.

Consider the following snippets for examples of how to implement this change:

Kotlin Groovy

build.gradle

```groovy
plugins {
    id("java")
}

tasks.named('myRunTask', JavaExec) {
    jvmArgumentProviders.clear() // Clear existing JVM argument providers
    maxHeapSize = null // Clear max heap size
    jvmArgs = ["-Dfoo", "-Dbar"] // Set new JVM arguments
}
```

Deprecation of `archives` configuration

The `archives` configuration added by the `base` plugin has been deprecated and will be removed in Gradle 10.0.0. Adding artifacts to the `archives` configuration will now result in a deprecation warning.

If you want the artifact to be built when running the `assemble` task, add the artifact (or the task that produces it) as a dependency on `assemble`:

build.gradle.kts

```kotlin
val specialJar = tasks.register<Jar>("specialJar") {
    archiveBaseName.set("special")
    from("build/special")
}

tasks.named("assemble") {
    dependsOn(specialJar)
}
```

Deprecation of the `Configuration.visible` property

Prior to Gradle 9.0.0, any configuration with `isVisible()` returning `true` would implicitly trigger artifact creation when running the `assemble` task. This behavior was removed in Gradle 9.0.0, and the `Configuration.visible` property no longer has any effect. The property is now deprecated and will be removed in Gradle 10.0.0. You can safely remove any usage of `visible`.

If you want the artifacts of a configuration to be built when running the `assemble` task, add an explicit task dependency on `assemble`:

build.gradle.kts

```kotlin
val specialJar = tasks.register<Jar>("specialJar") {
    archiveBaseName.set("special")
    from("build/special")
}

configurations {
    consumable("special") {
        outgoing.artifact(specialJar)
    }
}

tasks.named("assemble") {
    dependsOn(specialJar)
}
```

Deprecation of non-string `projectProperties` in `GradleBuild` task

The `GradleBuild` task now deprecates using non-String values in `startParameter.projectProperties`. While the type is declared as `Map<String, String>`, there was no strict enforcement, allowing non-String values to be set. This deprecated behavior will be removed in Gradle 10.0.0.

If you are using non-String values in project properties, convert them to `String` representation:

Kotlin Groovy

build.gradle

```groovy
def myIntProp = 42

tasks.register('nestedBuild', GradleBuild) {
    startParameter.projectProperties.put('myIntProp', "$myIntProp") // Convert int to String
}
```

Deprecation of project properties for toolchain configuration

In previous versions of Gradle, you could configure toolchains using project properties on the command line with the `-P` flag. For example, to disable toolchain auto-detection, you could use `-Porg.gradle.java.installations.auto-detect=false`. This behavior is deprecated and will be removed in Gradle 10.0.0. Instead, you should specify these settings as Gradle properties using the `-D` flag:

```text
-Dorg.gradle.java.installations.auto-detect=false
```

© 2025 Gradle, Inc. Gradle®, Develocity®, Build Scan®, and the Gradlephant logo are registered trademarks of Gradle, Inc.